In-Depth Study Notes: Oracle SQL Data Types and Advanced Referential Integrity

As an expert Oracle SQL Developer, mastering physical data types and relationship constraints ensures optimal disk storage and guarantees that your relational models do not collapse under orphaned data. Based on the provided execution logs (CH-10 ORACLE DATA TYPES.TXT and CH10-3.TXT), these notes break down the architecture of **Data Storage Types**, **Metadata Views**, **Constraint Naming Conventions**, and **Parent-Child Deletion Rules**.

Concept 1: Character and Numeric Data Types

Core Purpose: When defining a table schema, developers must explicitly declare exactly what kind of data each column will store, and how much physical disk space to allocate. Choosing the wrong data type leads to severe performance degradation and wasted RAM.

Keyword Breakdown:

- **CHAR(size)**: Stores fixed-length character strings 1. If you insert 'M' into a CHAR(10) column, Oracle literally pads the remaining 9 bytes with blank spaces. It has a maximum limit of 2,000 bytes 1.
- **VARCHAR2(size)**: An Oracle-specific, variable-length data type 2. It only consumes the bytes required by the actual data; inserting "MADHUSUDAN" into a VARCHAR2(10) column only uses 10 bytes, saving memory 2. It has a maximum limit of 4,000 bytes 2.
- **NVARCHAR2(size)**: Used to store multi-byte Asian languages or specialized Unicode 2.
- **NUMBER(precision, scale)**: Stores positive/negative integers, decimals, and exponents, handling up to 38 digits of precision 3.

Comparison of String Data Types:

Data Type,Sizing Requirement,Maximum Size,Memory Behavior
CHAR,Optional (Defaults to 1) 1,"2,000 Bytes 1",Padded. Wastes space if data is shorter than column limit.
VARCHAR,Mandatory 1,"4,000 Bytes","ANSI/SQL 1999 Compliant. Available in MySQL, SQLServer, DB2, etc 1."
VARCHAR2,Mandatory 2,"4,000 Bytes 2",Oracle Specific. Highly efficient variable length 2.

Syntax Breakdown:

```
CREATE TABLE table_name (
    col_1 VARCHAR2(max_bytes),
    col_2 CHAR(exact_bytes),
    col_3 NUMBER(total_digits, decimal_places)
);
```

Detailed Example: An enterprise needs to store basic employee profile information. They require a fixed 1-character flag for gender, a variable-length string for the name, and a standard number for their salary.

```
CREATE TABLE EMP_PROFILE (
    GENDER CHAR(1),
    NAME VARCHAR2(50),
    SALARY NUMBER(10, 2)
```

```
);
```

Execution Logic: When parsing the DDL, Oracle reserves exactly 1 byte per row for CHAR(1) on the database block 1. For VARCHAR2, it sets up a dynamic pointer mechanism that only writes the exact length of the inputted string to the data block 2.

Pro-Tips/Best Practices:

- **Never use CHAR for Names:** Only use CHAR for data that is universally fixed in length (e.g., 'Y'/'N' flags, state abbreviations like 'NY'). For everything else, strictly use VARCHAR2 to prevent trailing whitespace errors during WHERE clause evaluations.

Concept 2: Temporal Types, Intervals, and Large Objects

Core Purpose: Modern databases must track complex chronological events across multiple global timezones, calculate durations, and store massive binary files like images or PDFs directly tied to database records.

Keyword Breakdown:

- **DATE**: Requires exactly 7 bytes and stores century, year, month, day, hour, minute, and second 3. It cannot capture fractions of a second 3.
- **TIMESTAMP**: Stores everything DATE does, but adds fractional seconds (up to 9 digits of accuracy) 4.
- **TIMESTAMP WITH TIME ZONE**: Additionally stores the physical timezone offset (e.g., +5.30 for India, -4.30 for Canada) 4.
- **TIMESTAMP WITH LOCAL TIME ZONE**: Stores data in the server's timezone, but automatically translates the output to match the request user's local timezone (ideal for global MNCs) 5.
- **INTERVAL**: Stores a specific duration of time, rather than a point in time (e.g., INTERVAL YEAR TO MONTH for an LIC Policy duration, or INTERVAL DAY TO SECOND for journey durations) 5, 6.
- **BFILE**: Stores a locator (pointer) to an external file resting on the operating system, allowing access to files up to 4GB in size like movies, songs, or photographs 6.

Comparison of Date vs. Timestamp:

Temporal Type,Fractional Seconds?,Timezone Aware?,Business Use Case
DATE,No,No,"Employee Hire Dates, Birthdays."
TIMESTAMP,Yes (0-9 precision) 4,No,High-frequency stock market or ATM transactions 4.
TIMESTAMP WITH TIME ZONE,Yes,Yes 4,International flight departure/landing schedules 4.

Syntax Breakdown:

```
CREATE TABLE table_name (
    col_1 TIMESTAMP(fractional_precision) WITH TIME ZONE,
    col_2 BFILE
);
```

Execution Logic: For a BFILE, the Oracle Engine does not actually ingest the massive 4GB movie file into the database tablespace. It simply stores an external file directory path. This keeps the database lightweight while allowing query access to the heavy asset 6.

Concept 3: User-Defined Constraint Naming and Metadata

Core Purpose: By default, Oracle generates system names for constraints (e.g., **SYS_C0069933**) 7, 8. When an insert fails, an error like "SYS_C0069933 violated" tells the developer absolutely nothing about what went wrong. Explicitly naming constraints resolves this issue.

Keyword Breakdown:

- **CONSTRAINT**: The keyword used to declare a custom name for a database rule before defining the rule itself.
- **USER_CONSTRAINTS**: A Data Dictionary view containing metadata about the constraint rules 9.
- **USER_CONS_COLUMNS**: A Data Dictionary view mapping those constraints to their specific columns 10.

Syntax Breakdown:

```
CREATE TABLE table_name (
    column_name data_type CONSTRAINT custom_name CONSTRAINT_TYPE
);
```

Detailed Example: An application requires tracking PAN cards and Aadhaar cards. Both must be unique. Instead of letting Oracle name them randomly, we explicitly name them using the standard formula: USERNAME_TABLENAME_COLUMNNAME_CONSTRAINTTYPE 8.

```
CREATE TABLE PINFO (
    NM VARCHAR(10),
    PANCARD VARCHAR(10) CONSTRAINT SCOTT_PINFO_PANCARD_UK UNIQUE NOT
NULL,
    ADHARCARD VARCHAR(14) CONSTRAINT SCOTT_PINFO_ADAHRCARD_UK UNIQUE
NOT NULL
);
```

Execution: If a duplicate Aadhaar is inserted, the engine throws a readable error: ***ORA-00001: unique constraint (SCOTT.SCOTT_PINFO_ADAHRCARD_UK) violated*** *11.*

Pro-Tips/Best Practices:

- **Standard Suffixes:** Always use standardized 2-letter suffixes for your custom constraint names so DBAs know immediately what they are dealing with: PK (Primary Key), UK (Unique Key), CK (Check), RK or FK (Foreign Key / References), NN (Not Null) 11.

Concept 4: Foreign Key Deletion Rules (ON DELETE)

Core Purpose: When two tables are linked (a Parent and a Child), deleting a row in the Parent table creates a massive risk: what happens to the linked Child rows? Oracle provides three strict relational rules to handle these orphaned records 12.

Keyword Breakdown:

- **NO ACTION (Default)**: The database violently rejects the deletion of the Parent row if any Child rows exist 12, 13.
- **ON DELETE CASCADE**: The database implicitly deletes all linked Child rows first, and then deletes the Parent row 12, 14.

- **ON DELETE SET NULL**: The database nullifies the Foreign Key column on the Child rows, severing the link, and then deletes the Parent row 12, 15.

Comparison of Deletion Behaviors:

Foreign Key Rule,Action when Parent is Deleted,Consequence on Child Table
NO ACTION,Throws ORA-02292: child record found 14.,Child rows are completely protected.
CASCADE,Succeeds immediately 15.,Child rows are permanently destroyed 12.
SET NULL,Succeeds immediately 16.,"Child rows are kept, but orphaned with a NULL foreign key 12."

Syntax Breakdown:

```
CREATE TABLE child_table (
    col_1 data_type,
    fk_col data_type REFERENCES parent_table(pk_col) [ON DELETE
CASCADE | ON DELETE SET NULL]
);
```

Detailed Example: We have a Parent P2 table and a Child C2 table configured with CASCADE.

```
CREATE TABLE P2 (NO NUMBER PRIMARY KEY , NM VARCHAR(10));
CREATE TABLE C2 (NO NUMBER REFERENCES P2 ON DELETE CASCADE, AMT
NUMBER);
```

If we execute DELETE P2 WHERE NO = 2;, Oracle intercepts the command 15, 17, 18.

Execution Logic: Before the engine deletes NO = 2 from the Parent P2 table, it scans the Child C2 table. Because ON DELETE CASCADE is active, it automatically executes a silent background deletion of any row in C2 where NO = 2 12, 14. Once the child records are wiped, it finalizes the deletion of the parent record 12.

Pro-Tips/Best Practices:

- **Extreme Caution with CASCADE:** ON DELETE CASCADE is highly dangerous in production environments. Deleting a single "Company" record could accidentally trigger the silent deletion of thousands of "Employee", "Invoice", and "Order" records chained to it. Prefer NO ACTION for critical financial data to force manual review before deletion.